SYSTEM DESIGN DOCUMENT

REVA Internship Opportunities (RIO) | School of Computer Science and Engineering, REVA University, Bengaluru

Particulars	Details
Task	Task 3 (SL 3), Project Track: System Design of the Module
Name of Student	[Your Name]
University Seat Number (USN)	[Your USN]
Project Title	ForgeGuard: An AI-Based Document Forgery Detection and Verification System
Track	[Track]
Centre	[Centre]
Faculty Guide	[Faculty Guide]
Date of Submission	20 October 2026

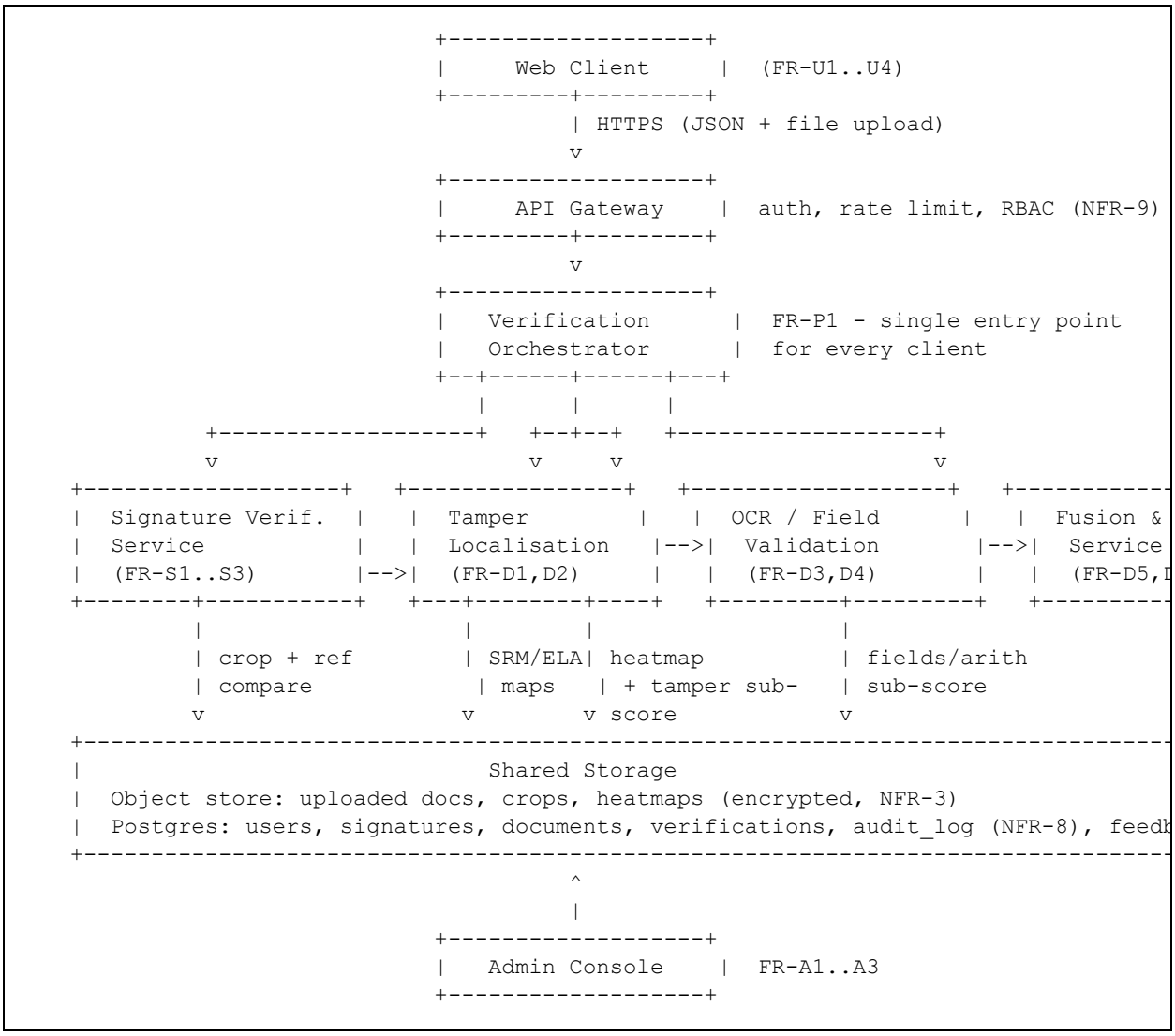
1. Introduction

1.1 Purpose. This document designs ForgeGuard from the requirements fixed in the Literature Survey and Requirement Analysis Report. Every component below exists to satisfy one or more functional requirements (FR-U, FR-S, FR-D, FR-A, FR-P) or non-functional requirements (NFR-1 to NFR-10) from that report, and the identifiers are carried over unchanged so the two documents trace against each other.

1.2 Design goals. Three decisions shape the whole design, each a direct response to a literature gap: (a) no single end-to-end classifier — signature, tamper and OCR evidence are produced by independent services and combined only at a fusion stage (Gap 5, FR-D5); (b) every detector sits behind one fixed interface (image/crop in, score and evidence out) so a model can be swapped without touching the orchestrator or the other detectors (Gap 8, NFR-7); (c) every stored verification is explainable and auditable, not just a label (Gap 6 and Gap 9, FR-D6, FR-A3, NFR-8).

2. Architecture Overview

ForgeGuard is a single backend service exposed through one REST API (FR-P1), fronted by a web client, with four detector workers and a fusion stage orchestrated per request. The assigned module, Document Tamper Detection and Localisation, is one of the four workers.



2.1 Why this shape. The orchestrator calls the four workers in the fixed order shown (signature and tamper run independently, OCR can run in parallel with either, fusion runs last), because FR-D5 requires all three evidence streams before a score is produced. Each worker is a separate process behind the same request/response contract, so NFR-7 is met by construction: replacing the tamper localiser with a newer model only changes that one box.

3. Module Design

3.1 Signature Verification Service (FR-S1, FR-S2, FR-S3, NFR-6)

Input: the full page image and the claimed signer's ID. Steps: (1) detect the signature region (object detector, following paper [13]); (2) clean the crop — remove ruled lines, stamps and background (FR-S2); (3) embed the crop and the signer's one-to-five stored references with a writer-independent Siamese/metric-learning encoder (papers [3], [6], [12]) and threshold the distance. Output: a similarity

score in [0, 1], a genuine/forged decision, and the cropped region's coordinates. No retraining is needed to add a new signer (NFR-6) — only new reference embeddings are stored.

3.2 Document Tamper Detection and Localisation (assigned module — FR-D1, FR-D2, FR-D6, NFR-2, NFR-10)

Input: the deskewed, resized, normalised page (FR-D1). Two streams run in parallel and are fused in an encoder-decoder, following SRM-Net (paper [49]): a fixed Spatial Rich Model (SRM) stream that captures noise-residual artefacts invisible to the eye, and a semantic RGB stream (ResNet/U-Net style) that reads layout and content. ELA is computed as one supporting channel only, not the sole signal (Gap 4). Output: a pixel-level tamper mask and heatmap, the largest suspicious regions with coordinates (FR-D6), and a 0–100 tamper sub-score passed to fusion. Trained with augmentation for rotation, scaling, noise, blur, recompression and print-scan (NFR-10); sized to run on one consumer GPU or CPU within the per-page time budget (NFR-2).

3.3 OCR / Field Validation Service (FR-D3, FR-D4)

Input: the page image and the declared document type. Steps: (1) OCR extraction (consensus across engines for financial documents, following FORENSIQ, paper [39]); (2) regex/format checks on names, dates and ID numbers (FR-D3); (3) for financial documents, an arithmetic gate checking total = net + tax (FR-D4, paper [39]). Output: per-field pass/fail flags and an OCR sub-score.

3.4 Fusion & Scoring Service (FR-D5, FR-D6)

Input: the three sub-scores (signature similarity, tamper score, OCR/field score) and their supporting evidence. A weighted rule (not a black-box classifier, Gap 5) combines them into one authenticity score 0–100 and a label: Genuine (≥ 80), Suspicious (40–79) or Forged (< 40) — thresholds configurable without a redeploy. Output: the final score, label, the tamper heatmap, per-check sub-scores and a templated plain-language reason (FR-D6, NFR-4), e.g. "Flagged Suspicious: signature similarity 0.41 (below 0.6 threshold); tamper region detected near the date field."

3.5 Admin & Audit (FR-A1, FR-A2, FR-A3, NFR-8, NFR-9)

Separate role (NFR-9) that can review documents labelled Suspicious, record a final human decision (stored as labelled feedback for retraining, FR-A2), and view analytics over the append-only audit log (FR-A3). Every verification request and every admin decision is written once to `audit_log` and never updated in place (NFR-8).

4. Data Design

Relational store (Postgres) for structured records; object storage for files, referenced by key. Simplified schema:

users	id, email, password_hash, role (verifier | admin), created_at
signatures	id, user_id (signer), image_key, embedding, created_at — 1 to 5 rows per signer (FR-U4)

documents	id, uploaded_by, doc_type, file_key, sha256_hash (FR-P2), uploaded_at
verifications	id, document_id, signature_sub_score, tamper_sub_score, ocr_sub_score, final_score, label, heatmap_key, reason_text, created_at
audit_log	id, actor_id, action, verification_id, detail_json, created_at — append-only (NFR-8)
feedback	id, verification_id, admin_id, final_decision, created_at — used for retraining (FR-A2)

5. API Design (FR-P1)

One REST API serves the web client today and any future client tomorrow:

- `POST /auth/register`, `POST /auth/login` — FR-U1
- `POST /signatures` (signer_id, image) — register a reference signature, FR-U4
- `POST /documents` (file, doc_type) → `202 {document_id}` — upload, FR-U2
- `GET /documents/{id}/verification` → score, label, heatmap URL, sub-scores, reason — FR-U3, FR-D5, FR-D6
- `GET /documents/{id}/report.pdf` — downloadable report, FR-U3
- `GET /verifications?user_id=` — history, FR-U3
- `POST /admin/verifications/{id}/decision` — FR-A2 (admin role only, NFR-9)
- `GET /admin/audit-log`, `GET /admin/analytics` — FR-A3

Upload returns immediately with a document ID (FR-D1–D5 run asynchronously within the NFR-1 budget); the client polls or receives a webhook/SSE push when the verification completes.

6. Verification Sequence (per upload)

1. Client calls `POST /documents`; gateway checks auth/role and forwards to the orchestrator.
2. Orchestrator stores the file, computes its SHA-256 (FR-P2), and pre-processes it: deskew, resize, normalise, SRM + ELA maps (FR-D1).
3. Signature service and tamper localiser run (in parallel, since neither depends on the other); OCR/field service runs concurrently with both.
4. Each service returns its sub-score and evidence to the orchestrator.
5. Fusion service combines the three sub-scores into the final score, label and reason (FR-D5, FR-D6).
6. Orchestrator writes the `verifications` row and an `audit_log` entry (NFR-8), then notifies the client.
7. If the label is Suspicious, the document additionally appears in the admin review queue (FR-A2).

7. Non-Functional Design Decisions

NFR	Design decision

NFR-1 (<10 s/page)	Signature, tamper and OCR run concurrently, not in sequence; each model is sized to finish its share of the budget.
NFR-2 (consumer GPU/CPU)	Tamper localiser uses ResNet-18-scale backbones, not large transformers; quantised inference path for CPU-only deployment.
NFR-3 (encryption, retention)	Object store server-side encryption; TLS in transit; scheduled job purges files past the configured retention window, keeping only hashes and scores.
NFR-4 (explainable)	Fusion output always includes the heatmap, the three sub-scores and a templated reason string — never a bare label.
NFR-5 (proper evaluation)	Each service reports precision/recall/F1/AUC on held-out splits in its own eval harness; class weighting applied where training data is imbalanced.
NFR-6 (few-shot signer)	Metric-learning embedding model is frozen at inference; adding a signer is an insert into `signatures`, not a training job.
NFR-7 (replaceable detectors)	Every worker implements the same contract: image/crop in, {sub_score, evidence} out, over an internal queue/HTTP call — swappable independently.
NFR-8 (audit log)	`audit_log` is insert-only at the DB level (no UPDATE/DELETE grants on that table for the application role).
NFR-9 (RBAC)	Gateway checks `role` claim on every request; admin-only routes return 403 for the verifier role.
NFR-10 (robust capture)	Training-time augmentation (rotation, scale, noise, blur, recompression, print-scan) across all three detector models.

8. Technology Stack

- API & orchestrator: Python (FastAPI) — async I/O suits the fan-out/fan-in call pattern in Section 6.
- Detector services: PyTorch (signature embedding model, SRM/RGB dual-stream localiser); Tesseract/PaddleOCR for text extraction.
- Storage: PostgreSQL for structured data; S3-compatible object storage for images/heatmaps.
- Web client: React, calling only the REST API in Section 5 (FR-P1), so a mobile client can reuse it unchanged.

9. Phase 2 (Out of Scope Here)

Consistent with the survey's scope notes (Section 4.4 of the Literature Survey Report): blockchain anchoring of the document hash already stored under FR-P2, and an anomaly-detection worker for forgery types outside the training distribution (Gap 8), can be added behind the same replaceable-detector interface (NFR-7) without changing the orchestrator, fusion logic or API.